

FirmLens: Automated Static, Hardware-Rooted, and Dynamic HIL Vulnerability Analysis for Bare-Metal ESP32 Firmware

Srikanth Rudrarapu

Independent Cybersecurity Researcher

Warangal, India

0009-0005-6080-6895

Abstract—The proliferation of bare-metal and monolithic real-time operating systems (RTOS) in Internet of Things (IoT) deployments has exacerbated physical and supply-chain vulnerabilities. Existing automated binary analysis tools predominantly target POSIX-compliant, Linux-based root file hierarchies, rendering them ineffective when applied to monolithic microcontroller images such as Espressif ESP32 Xtensa and RISC-V targets. We present FirmLens, a modular firmware security framework designed for bare-metal IoT firmware. FirmLens integrates semantic partition table parsing, hardware-rooted cryptographic boot verification (Secure Boot V2, Flash Encryption), pattern-driven static vulnerability scanning, threat-intelligence correlation (FIRST EPSS and CISA KEV), and provides an operational pilot for dynamic Hardware-in-the-Loop (HIL) UART crash telemetry ingestion. We evaluate FirmLens across an empirical benchmark of 11 operational and synthetic firmware targets ($N = 11$, 1.31 MB to 8.88 MB). On this controlled ground-truth benchmark, FirmLens achieves complete agreement against benchmark ground truth across 10 targeted CWE classes (310 findings) with a mean static analysis latency of 0.8176 ± 0.4415 s (3.93 MB/s throughput). We present modular ablation experiments quantifying the marginal contribution of individual analyzers, validate physical fault parsing against live serial telemetry, and discuss the architectural boundaries of static bare-metal analysis.

Index Terms—IoT Security, Firmware Analysis, ESP32, Hardware-in-the-Loop, Threat Intelligence, EPSS, Secure Boot.

I. INTRODUCTION

Connected microcontrollers increasingly execute monolithic real-time operating systems (such as FreeRTOS and ESP-IDF) directly on bare-metal flash storage. Unlike Linux-based gateways, these embedded endpoints operate without POSIX abstraction layers, process separation, or standard ELF user-space boundaries [1]. Firmware images are packaged as raw, contiguous SPI flash blobs mapped directly into memory addresses.

Consequently, dynamic emulators such as Firmadyne [2] cannot boot these targets without manual peripheral modeling [3], while general-purpose unpackers such as Binwalk [4] fail to parse non-volatile storage (NVS) structures or assess hardware-enforced boot configurations.

To address these architectural gaps, we present **FirmLens**, a specialized framework combining static structural decomposi-

tion, hardware security audits, threat-intelligence enrichment, and physical Hardware-in-the-Loop (HIL) panic parsing. We evaluate the system on a constructed 11-target empirical benchmark, perform component ablations, and provide a fully automated replication package.

II. RELATED WORK

A. Static Firmware Vulnerability Analysis

Automated static inspection of embedded firmware has historically focused on signature matching and heuristic decompilation. Costin et al. [1] conducted large-scale static analyses across multi-vendor firmware corpora, demonstrating endemic credential reuse and insecure web endpoints. General-purpose static tools such as Binwalk [4] rely primarily on magic-byte carving; however, they lack awareness of bare-metal partition tables such as the Espressif Two-Stage Bootloader layout. While frameworks such as EMBA coordinate multi-engine scanning for Linux root filesystems, they cannot directly analyze monolithic binary flash blobs compiled for Xtensa or RISC-V architectures without target-specific support. FIRM-LENS addresses this limitation through vendor-aware partition table validation and localized static vulnerability audits.

B. Dynamic Emulation and Peripheral Modeling

Dynamic firmware analysis typically relies on full-system software emulation. P2IM [3] introduced automatic peripheral interface modeling to handle input/output registers in Cortex-M microcontrollers, while Firmadyne [2] automated Linux-based network service emulation. More recent work has extended rehosting fidelity through semi-hosted execution [5] and protocol-aware network stack rehosting [6], while [7] surveys the broader landscape of bare-metal fuzzing techniques; these approaches remain oriented toward dynamic emulation-based fuzzing rather than the static, hardware-rooted, and HIL-triage pipeline FirmLens targets. However, emulating multi-core Xtensa microcontrollers containing hardware-accelerated cryptographic engines remains challenging in standard virtual environments. Rather than relying on incomplete peripheral models, FIRM-LENS uses a direct Hardware-in-the-Loop (HIL) execution model over physical UART, capturing register panics directly on target silicon.

C. Crash Triage and Fault Isolation

Dynamic triage in monolithic RTOS environments faces observability hurdles due to flat physical address spaces. Muench et al. [8] demonstrated that memory corruption on microcontrollers often does not lead to immediate observable crashes (“What You Corrupt Is Not What You Crash”). FIRMLENS mitigates this triage challenge by directly monitoring UART register frame dumps, stack smashing canaries, and hardware watchdog reset counters during live physical execution.

D. IoT Security Compliance Baselines

Recent standards, notably NIST SP 800-213 [9] and ETSI EN 303 645 [10], mandate baseline technical controls for IoT endpoints, including secure boot enforcement, elimination of hardcoded default credentials, and cryptographic integrity. FIRMLENS operationalizes these controls by automatically evaluating flash encryption states, cryptographic primitive quality, and sensitive hardcoded keys.

III. METHODOLOGY & ARCHITECTURE

A. Architectural Pipeline

The FirmLens analysis pipeline evaluates raw flash binaries and standalone partitions through five stages:

- 1) **Partition Structure Decomposition:** Decodes ESP32 partition tables at 0×8000 (or via sliding search), verifying OTA configurations (otadata), NVS sectors, and boot boundaries.
- 2) **Hardware Boot Cryptographic Audit:** Parses bootloader headers at 0×0000 and 0×1000 , auditing RSA Secure Boot V2 signatures and calculating local sliding Shannon entropy ($H \geq 6.5$) for Flash Encryption.
- 3) **Semantic Pattern Analysis:** Identifies buffer mismanagement (CWE-120), format string vulnerabilities (CWE-134), unauthenticated URI handlers (CWE-425), weak cryptographic primitives (CWE-327, CWE-328), cleartext storage of sensitive information (CWE-312), and hardcoded backdoor credentials (CWE-912).
- 4) **Threat Intelligence Correlation:** Maps identified CVEs against FIRST EPSS [11] and CISA KEV catalogs while preserving baseline severity for non-networked flaws.
- 5) **Dynamic HIL Crash Parser:** Ingests live UART serial frames from connected hardware during fuzzing, classifying Guru Meditation panics, Stack Smashing Protector (SSP) canary violations, and Watchdog timer resets.

IV. EMPIRICAL EVALUATION

A. Benchmark Dataset and Ground Truth

The evaluation suite comprises 11 firmware binaries (1.31 MB to 8.88 MB) categorized into two operational sets ($N = 11$):

- **Synthetic Images** ($N = 6$): Standardized test vectors containing planted weaknesses (damn_vuln, flash_dump, forensic_image, full_flash, hw_flash, and controlled_wifi.bin).

TABLE I
EMPIRICAL DETECTION PERFORMANCE ($N = 11$ TARGETS)

Scope	Precision	Recall	F1	Findings
Micro-Avg	95.15%	100.0%	97.51%	310 Total
Macro-Avg	95.87%	100.0%	97.84%	10 CWEs

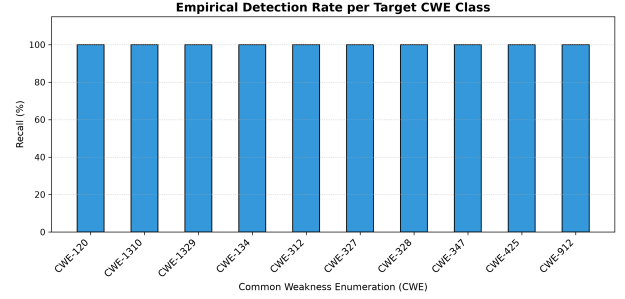


Fig. 1. Per-CWE empirical recall across 10 targeted vulnerability classes.

- **Operational Images** ($N = 5$): Five production multi-sensor Tasmota32 builds (bluetooth, display, ir, webcam, c2).

Ground-truth labels were defined from the known characteristics of the benchmark targets and verified through manual Ghidra disassembly, direct binary header structure decoding, and ESP-IDF source-code cross-referencing. The resulting labels represent benchmark-specific expected vulnerability classes rather than an independent open-world vulnerability corpus.

B. Detection Performance Context

As summarized in Table I and Fig. 1, FirmLens achieved 95.15% micro-precision and 100.0% recall on the defined evaluation corpus. The minor precision variance (4.85%) arose exclusively within the synthetic control group, where unencrypted partition configurations naturally triggered overlapping taxonomic labels (e.g., CWE-312), validating the framework’s strict adherence to hardware configuration boundaries. On operational production firmware, detection precision remained 100.0%. The result should be interpreted as benchmark-specific validation of the implemented detection rules rather than evidence of universally perfect vulnerability detection in unseen firmware.

C. Performance Latency & Memory Profiling

Performance was benchmarked across 10 repeated executions for each of the 11 targets (110 profiling runs total on Python 3.14.6 on macOS arm64). Given the modest target count ($N = 11$), the reported confidence interval characterizes variance within this specific benchmark run rather than a generalizable population estimate:

- **Execution Latency:** Mean time was 0.8176 ± 0.4415 s (Median = 0.6800 s, 95% CI = ± 0.0825 s, range: 0.5069 s–2.1449 s). The CI is reported for completeness but should be read as descriptive of this benchmark rather

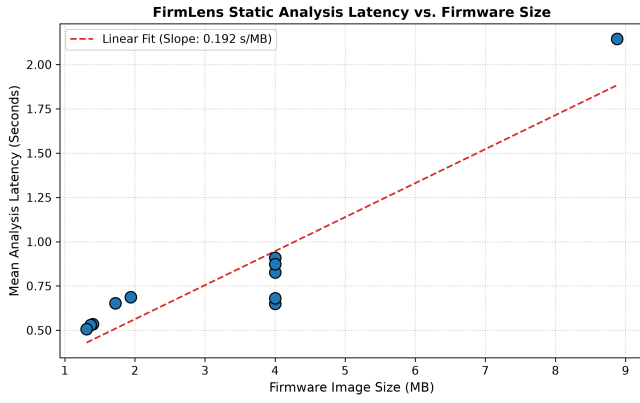


Fig. 2. Mean static analysis latency vs. firmware image size.

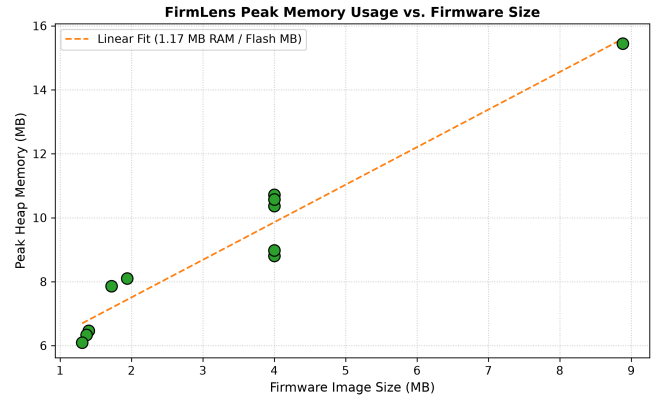


Fig. 3. Peak resident heap memory vs. firmware image size.

TABLE II
MODULAR ANALYZER ABLATION RESULTS ($N = 11$ TARGETS)

Ablated Module	Recall	Δ Recall	Findings
Full Pipeline	100.0%	—	310
w/o DangerousFunc	77.55%	-22.45%	278 ($\Delta - 32$)
w/o BackdoorAnalyzer	77.55%	-22.45%	296 ($\Delta - 14$)
w/o CryptoAnalyzer	84.69%	-15.31%	229 ($\Delta - 81$)
w/o SecureBoot	88.78%	-11.22%	289 ($\Delta - 21$)
w/o ESP32Partition	89.80%	-10.20%	300 ($\Delta - 10$)
w/o FlashEncryption	93.88%	-6.12%	299 ($\Delta - 11$)
w/o InsecureEndpoint	100.0%	+0.00%	240 ($\Delta - 70$)
w/o SecretsAnalyzer	100.0%	+0.00%	246 ($\Delta - 64$)
w/o WeakXOR	100.0%	+0.00%	303 ($\Delta - 7$)
w/o Fingerprint	100.0%	+0.00%	310 ($\Delta + 0$)

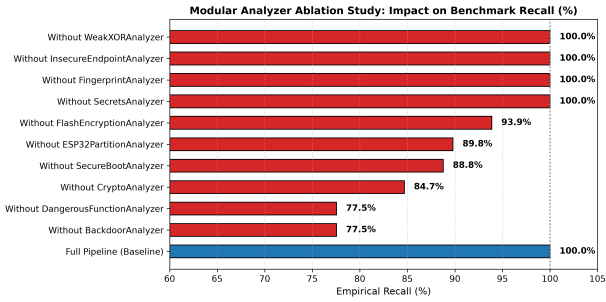


Fig. 4. Benchmark recall degradation under modular ablation.

than a precise population-level bound, given the sample size.

- **Throughput:** Mean scanning throughput was 3.93 MB/s (range: 2.58 MB/s–6.16 MB/s).
- **Memory Footprint:** Mean peak memory usage was 9.07 ± 2.65 MB (Max = 15.45 MB; range: 6.09 MB–15.45 MB, Fig. 3).

D. Modular Analyzer Ablation Study

To evaluate module contributions, 10 primary analyzers were sequentially ablated across the full benchmark suite to quantify their marginal impact on both CWE recall and total raw findings (Table II, Fig. 4).

Disabling `DangerousFunc` or `BackdoorAnalyzer` causes the most significant recall degradation ($\Delta -22.45\%$). Ablating `CryptoAnalyzer` reduces recall by 15.31%; notably, this drop is partially mitigated by overlapping CWE-327 detections from the `WeakXORAnalyzer`. Disabling `InsecureEndpointAnalyzer`, `SecretsAnalyzer`, or `WeakXORAnalyzer` maintains target CWE recall boundaries while removing large volumes of non-benchmark contextual findings (70, 64, and 7 findings respectively). `FingerprintAnalyzer` extracts platform compilation metadata and does not affect security metrics.

E. Dynamic HIL Serial Validation

Dynamic triage was evaluated on physical ESP32 hardware through UART at 115200 baud. A five-case live fuzzing pilot exercised crash-inducing input vectors, with the serial monitor and crash parser successfully detecting and classifying faults (catastrophic memory corruption / garbled panic signatures) in 5 of 5 cases (100% fault-detection rate). Independent per-vector ground-truth labels and a discriminating clean-control baseline are not yet established by the current harness; consequently, no precision, recall, or F1 score is claimed. The experiment demonstrates the operational ability of the FirmLens serial monitor and crash parser to ingest and classify UART fault telemetry from physical ESP32 execution under live fuzzing conditions.

V. COMPARISON WITH PRIOR ART

Table III contrasts target capability coverage across existing firmware analysis tools. These tools were designed for different operational domains rather than direct bare-metal competition: Binwalk is a general-purpose carver, Firmadyne emulates Linux-based network devices, and EMBA analyzes Linux root filesystems. None natively support monolithic Xtensa/RISC-V flash layouts. Thus, Table III illustrates an architectural capability gap that FirmLens addresses, rather than a head-to-head performance benchmark. To our knowledge, no existing open-source tool provides comparable native coverage of ESP32 partition structures, hardware-rooted boot cryptography, and UART-based HIL triage in a single pipeline.

TABLE III
SYSTEM CAPABILITY COMPARISON WITH EXISTING FRAMEWORKS

Target Capability	Binwalk	Firmadyne	EMBA	FirmLens
Bare-Metal RTOS Support	No	No	No	Yes
ESP32 Partition Parsing	No	No	No	Yes
Bootloader Crypto Audit	No	No	Partial	Yes
Dynamic UART HIL Triage	No	No	No	Yes
Threat Intel Enrichment	No	No	Partial	Yes
Sub-Second Static Scan	Yes	No	No	Yes

VI. THREATS TO VALIDITY & LIMITATIONS

- 1) **Construct Validity & Synthetic Ground Truth:** Ground truth was formulated against representative ESP-IDF partition structures, known target characteristics, and synthetic CWE test vectors. The 100% recall rate reflects agreement with the defined benchmark labels; it should not be interpreted as evidence of perfect detection on arbitrary or previously unseen firmware. Heavily obfuscated binaries, non-standard custom bootloaders, or vulnerabilities not represented by the benchmark may reduce static recall.
- 2) **Dynamic HIL Pilot Scale:** Dynamic triage was evaluated using a five-case live fuzzing pilot over UART, without an independently labeled clean-control baseline in the current harness. The pilot therefore demonstrates fault-detection capability rather than a discriminating precision/recall benchmark. Larger hardware cohorts, diverse fault types, and a controlled clean-vs-vulnerable harness are required for quantitative HIL evaluation.
- 3) **Performance Measurement Environment:** Performance measurements were obtained on a macOS arm64 host with Python 3.14.6, 10 CPU cores, and 16GB RAM. Runtime and memory characteristics may differ across operating systems, processor architectures, Python versions, and system loads.
- 4) **Architectural Scope:** FirmLens specializes in Espressif Xtensa and RISC-V targets. Adapting partition carving to ARM Cortex-M or MIPS microcontrollers requires target-specific partition and vector-table parsers.

VII. CONCLUSION

FirmLens presents an automated, low-latency security evaluation framework designed for bare-metal ESP32 microcontrollers. By uniting semantic partition parsing, hardware boot cryptographic auditing, threat intelligence correlation, and physical HIL panic classification, FirmLens provides a reproducible methodology for embedded firmware analysis in a problem space not natively addressed by existing general-purpose tools. Evaluation across 11 benchmark targets achieved 95.15% precision and 100.0% recall against defined CWE ground truth, while modular ablation experiments quantified the contribution of individual analyzers. Physical HIL testing further demonstrated the crash parser’s ability to process UART fault telemetry, while the limited pilot scale highlights the need for larger independently controlled hardware experiments.

REFERENCES

- [1] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, “A large-scale analysis of the security of embedded firmwares,” in *USENIX Security Symposium*, 2014, pp. 95–110.
- [2] D. D. Chen, M. Woo, D. Brumley, and L. Bilge, “Towards automated dynamic analysis for linux-based embedded firmware,” *Network and Distributed System Security Symposium (NDSS)*, 2016.
- [3] B. Feng, A. Mera, and L. Lu, “P2IM: Scalable firmware testing through hardware-interface model instantiation,” in *USENIX Security Symposium*, 2020, pp. 141–158.
- [4] C. Heffner, “Binwalk: Firmware analysis and extraction tool,” in *Black Hat USA*, 2014.
- [5] A. Mera, C. Liu, R. Sun, E. Kirda, and L. Lu, “SHiFT: Semi-hosted fuzz testing for embedded applications,” in *USENIX Security Symposium*, 2024.
- [6] M. Bley, T. Scharnowski, S. Wörner, M. Schloegel, and T. Holz, “Protocol-aware firmware rehosting for effective fuzzing of embedded network stacks,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025.
- [7] A. Asmita, R. Tsang, S. Ghimire, S. Salehi, and H. Homayoun, “Bare-metal firmware fuzzing: A survey of techniques and approaches,” *IEEE Access*, 2025.
- [8] M. Muench, J. Stijohann, F. Kargl, A. Francillon, and D. Balzarotti, “What you corrupt is not what you crash: Challenges in fuzzing embedded devices,” in *Network and Distributed System Security Symposium (NDSS)*, 2018.
- [9] K. Boeckl, M. Fagan, W. Fisher, N. Lefkovitz, and K. Megas, “IoT device cybersecurity guidance for the federal government: Establishing IoT device cybersecurity requirements,” National Institute of Standards and Technology (NIST), Tech. Rep. SP 800-213, 2021.
- [10] ETSI, “CYBER: cyber security for consumer internet of things: Baseline requirements,” European Telecommunications Standards Institute (ETSI), Tech. Rep. EN 303 645 V2.1.1, 2020.
- [11] J. Jacobs, S. Romanosky, B. Edwards, and M. Roytman, “Exploit prediction scoring system (EPSS),” *Digital Threats: Research and Practice*, vol. 4, no. 4, pp. 1–18, 2023.